

VVSS, Lab 5: Selenium Web UI Testing

Computer Science – 2025-2026/II

Objective

Write automated **web UI tests** using **Selenium WebDriver** for the FlavorFiesta Spring Boot application.

Background

FlavorFiesta is a web application where users can share recipes, post comments, and write reviews. It has a browser-based UI built with Thymeleaf templates. Read the **README** file for an overview of the application, its pages, and available HTML element IDs.

What to Do

1. **Choose 3 functionalities** from the application to test. Examples:
 - Login / Logout
 - Browsing the recipe list
 - Viewing a recipe's details
 - Posting a comment on a recipe
 - Navigating to a user profile
 - Accessing a non-existent page
2. **Write 2 Selenium tests per functionality** (6 tests total, across 3 test files):
 - **Happy path test** – tests the normal, expected behavior (e.g., login with correct credentials redirects to the home page).
 - **Negative path test** – tests an edge case or error scenario where the application should handle the situation correctly (e.g., login with wrong password stays on the login page). **Important:** this test should still **pass** – it verifies that the application handles the error correctly, not that something crashes.

Test stubs are provided in `src/test/java/.../example_for_homework/`:

- `Functionality1Test.java` – 2 tests for your first functionality
 - `Functionality2Test.java` – 2 tests for your second functionality
 - `Functionality3Test.java` – 2 tests for your third functionality
-

Example

A complete working example is provided in `ui/TestWebApp.java`. It tests the **login** functionality:

- `testSuccessfulLogin()` – enters correct credentials, verifies redirect to `/home` and that the sign-out button is visible.
- `testFailedLogin()` – enters wrong password, verifies the page stays on `/login` and the sign-out button is not present.

Use this as a reference for how to: - Start the application on a random port with `@SpringBootTest(webEnvironment = RANDOM_PORT)` - Create a `ChromeDriver` in `@BeforeAll` - Find elements with `driver.findElement(By.id(...))` - Wait for page loads with `WebDriverWait` and `ExpectedConditions` - Clean up test data and close the browser in `@AfterAll`

Bonus: Serenity BDD

For additional points, integrate **Serenity BDD** into your Selenium tests. Serenity provides: - Structured test reports - Step-by-step documentation of test execution - BDD-style test organization

Summary

Deliverable	Count
Functionalities tested	3
Tests per functionality	2 (happy path + negative path)
Total Selenium tests	6 (across 3 test files)
Bonus: Serenity BDD	optional